Walchand College of Engineering, Sangli

Robotics Laboratory — Assignment 6

Implementation of Orientation and Forward Kinematics for Dobot Magician

Name - Atharv Kulkarni

PRN - 245109074

Class - TY CSE(B)

GitHub Link of My:
Work:https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_7/Session4_Pick_and_Place(1).ipynb

https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_6/Session1_Orientation(1).ipynb

https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_6/Session2_Forward_Kinematics.ipynb

Experiment Title:

Implementation of Orientation and Forward Kinematics for Dobot Magician

AIM:

To study robot orientation and forward kinematics by initializing a robotic-arm simulation, examining its joints and coordinate frames, and observing how joint-angle configurations affect the end-effector position.

OBJECTIVES:

- To set up the Python simulation environment using Google Colab and PyBullet.
- To load the robotic arm and ground plane in the simulation environment.
- To identify the robot joints and understand the world frame and tool/end-effector frame.
- To inspect the initial end-effector position and test different joint-angle configurations.
- To observe the relationship between joint angles and the resulting end-effector pose.

THEORY:

Robot orientation describes the direction in which a robot link or end-effector is pointing. A coordinate frame provides a reference for expressing position and orientation. The world frame is fixed relative to the simulation environment, whereas the tool frame moves with the robot's end-effector.

Forward kinematics calculates the end-effector position and orientation from the known joint variables and the robot's link geometry. For a serial manipulator, the transformations associated with successive joints and links are combined to obtain the end-effector pose relative to the world frame. Changing a joint angle changes the configuration of the arm and may change the end-effector position and orientation.

EXPERIMENT STEPS WITH SCREENSHOT OF EACH STEP:

Step 1: Install and import the required libraries

Install PyBullet and import the required Python libraries, including PyBullet, PyBullet data, NumPy, Matplotlib, and time.

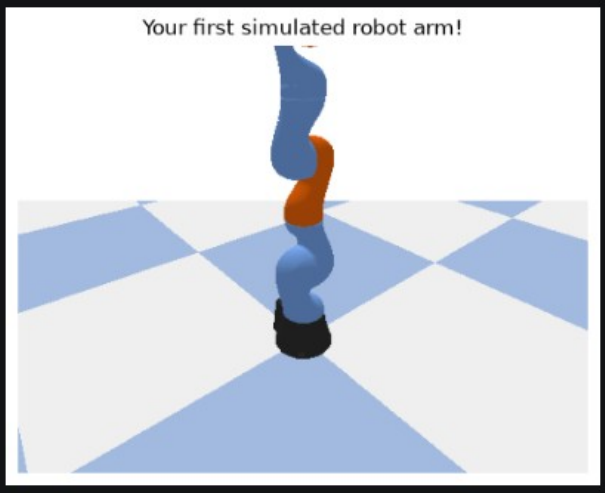
```
!pip install pybullet -q
import pybullet as p
import pybullet_data
import numpy as np
import matplotlib.pyplot as plt

client = p.connect(p.DIRECT)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.setGravity(0, 0, -9.8)
p.loadURDF("plane.urdf")
robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
for _ in range(20):
    p.stepSimulation()

view_matrix = p.computeViewMatrixFromYawPitchRoll((0, 0, 0.4), 1.4, 50, -30, 0, 2)
proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=480/360, nearVal=0.1, farVal=3.0)
img = p.getCameraImage(480, 360, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
rgb = np.reshape(img[2], (360, 480, 4))[:, :, :3]
plt.figure(figsize=(6, 4.5))
plt.imshow(rgb)
plt.axis('off')
plt.title("Your first simulated robot arm!")
plt.show()
```

✓ 1.6s

```
pybullet build time: Oct 15 2026 18:47:44
```



Setup (run these two cells first)

```
!pip install pybullet -q
```

✓ 0.7s

```
# One-time setup — run this first (takes ~15-30 seconds on Colab)

import pybullet as p
import pybullet_data
import numpy as np
import matplotlib.pyplot as plt
import time

print("PyBullet ready.")
```

✓ 0.5s

```
pybullet build time: Oct 15 2026 18:47:44
PyBullet ready.
```

Step 2: Initialize the simulation environment

Create the simulation client, set the asset search path and gravity, and prepare the ground plane.

```
plane_id = None

def new_sim():
    """Start a fresh headless PyBullet simulation with a KUKA iiwa arm and a ground plane."""
    global client, robot_id, plane_id
    if client is not None:
        p.disconnect(client)
    client = p.connect(p.DIRECT)  # headless — no local GUI needed
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane_id = p.loadURDF("plane.urdf")
    robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
    for _ in range(20):
        p.stepSimulation()
    return robot_id

def snapshot(cam_target=(0, 0, 0.4), cam_dist=1.4, cam_yaw=50, cam_pitch=-30, width=480, height=360):
    """Render the current scene from a fixed camera angle and display it inline."""
    view_matrix = p.computeViewMatrixFromYawPitchRoll(cam_target, cam_dist, cam_yaw, cam_pitch, 0, 2)
    proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=width/height, nearVal=0.1, farVal=3.0)
    img = p.getCameraImage(width, height, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(img[2], (height, width, 4))[:, :, :3]
    plt.figure(figsize=(6, 4.5))
    plt.imshow(rgb)
    plt.axis('off')
    plt.show()

def get_num_joints():
    return p.getNumJoints(robot_id)

def get_joint_info():
    info = []
    for i in range(p.getNumJoints(robot_id)):
        ji = p.getJointInfo(robot_id, i)
        info.append({"index": ji[0], "name": ji[1].decode(), "type": ji[2]})
    return info

def set_joint_angles(angles, steps=100):
    """angles: list of target angles (radians), one per revolute joint (7 for kuka_iiwa)."""
    for i, a in enumerate(angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()

def get_end_effector_position():
    """KUKA iiwa end-effector is link index 6."""
    state = p.getLinkState(robot_id, 6)
    return state[0]  # (x, y, z)

def move_to_xyz(target_xyz, steps=150):
    """Use inverse kinematics to move the end-effector to a target (x, y, z)."""
    joint_angles = p.calculateInverseKinematics(robot_id, 6, target_xyz)
    for i, a in enumerate(joint_angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()
    return get_end_effector_position()

def spawn_cube(position, half_extent=0.03, color=(1, 0, 0, 1)):
    col = p.createCollisionShape(p.GEOM_BOX, halfExtents=[half_extent]*3)
    vis = p.createVisualShape(p.GEOM_BOX, halfExtents=[half_extent]*3, rgbaColor=color)
    return p.createMultiBody(baseMass=0.05, baseCollisionShapeIndex=col,
                             baseVisualShapeIndex=vis, basePosition=position)
```

Step 3: Load the robotic arm and render the scene

Load the robot model into the simulation, advance the simulation, and capture a snapshot of the robot.



```
def pick_and_place(object_start, object_target, lift_height=0.15):
    """Full grasp-lift-move-release routine. Spawns a fresh cube each call."""
    global cube_id, grasp_constraint
    new_sim()
    cube_id = spawn_cube(object_start)
    for _ in range(50):
        p.stepSimulation()

    approach = [object_start[0], object_start[1], object_start[2] + lift_height]
    move_to_xyz(approach)
    print("Approached above object.")

    move_to_xyz(object_start)
    grasp_constraint = p.createConstraint(robot_id, 6, cube_id, -1, p.JOINT_FIXED,
                                          [0, 0, 0], [0, 0, 0], [0, 0, 0])
    print("Grasped object.")

    move_to_xyz(approach)
    print("Lifted.")

    target_approach = [object_target[0], object_target[1], object_target[2] + lift_height]
    move_to_xyz(target_approach)
    move_to_xyz(object_target)
    print("Moved to target and lowered.")

    p.removeConstraint(grasp_constraint)
    move_to_xyz(target_approach)
    print("Released and retracted.")
    snapshot()

print("Helpers loaded.")
```

✓ 0.0s

```
Helpers loaded.
```

Step 4: Inspect the robot joints

Retrieve and display the joint information, including joint indices, names, and joint types.

```python
client = None
robot_id = None
plane_id = None

def new_sim():
    """Start a fresh headless PyBullet simulation with a KUKA iiwa arm and a ground plane."""
    global client, robot_id, plane_id
    if client is not None:
        p.disconnect(client)
    client = p.connect(p.DIRECT)  # headless — no local GUI needed
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane_id = p.loadURDF("plane.urdf")
    robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
    for _ in range(20):
        p.stepSimulation()
    return robot_id

def snapshot(cam_target=(0, 0, 0.4), cam_dist=1.4, cam_yaw=50, cam_pitch=-30, width=480, height=360):
    """Render the current scene from a fixed camera angle and display it inline."""
    view_matrix = p.computeViewMatrixFromYawPitchRoll(cam_target, cam_dist, cam_yaw, cam_pitch, 0, 2)
    proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=width/height, nearVal=0.1, farVal=3.0)
    img = p.getCameraImage(width, height, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(img[2], (height, width, 4))[:, :, :3]
    plt.figure(figsize=(6, 4.5))
    plt.imshow(rgb)
    plt.axis('off')
    plt.show()

def get_num_joints():
    return p.getNumJoints(robot_id)

def get_joint_info():
    info = []
    for i in range(p.getNumJoints(robot_id)):
        ji = p.getJointInfo(robot_id, i)
        info.append({"index": ji[0], "name": ji[1].decode(), "type": ji[2]})
    return info

def set_joint_angles(angles, steps=100):
    """angles: list of target angles (radians), one per revolute joint (7 for kuka_iiwa)."""
    for i, a in enumerate(angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()

def get_end_effector_position():
    """KUKA iiwa end-effector is link index 6."""
    state = p.getLinkState(robot_id, 6)
    return state[0]  # (x, y, z)

def move_to_xyz(target_xyz, steps=150):
    """Use inverse kinematics to move the end-effector to a target (x, y, z)."""
    joint_angles = p.calculateInverseKinematics(robot_id, 6, target_xyz)
    for i, a in enumerate(joint_angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()
    return get_end_effector_position()

def spawn_cube(position, half_extent=0.03, color=(1, 0, 0, 1)):
    col = p.createCollisionShape(p.GEOM_BOX, halfExtents=[half_extent]*3)
    vis = p.createVisualShape(p.GEOM_BOX, halfExtents=[half_extent]*3, rgbaColor=color)
    return p.createMultiBody(baseMass=0.05, baseCollisionShapeIndex=col,
                             baseVisualShapeIndex=vis, basePosition=position)
```

IMPLEMENTATION WITH OUTPUT:

Implementation Code Screenshot

Step 4 — Try running a real simulation cell

The cell below is a taste of what you'll do in Session 1: it installs PyBullet, loads a simulated robot arm, and takes a snapshot so you can *see* it. Run it now with Shift+Enter and watch what happens — don't worry about understanding the code yet, that's what Session 1 is for.

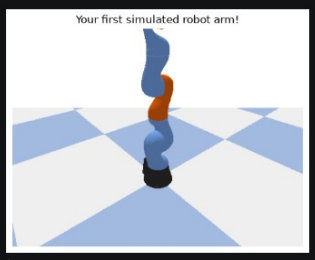
```
!pip install pybullet -q
import pybullet as p
import pybullet_data
import numpy as np
import matplotlib.pyplot as plt

client = p.connect(p.DIRECT)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.setGravity(0, 0, -9.8)
p.loadURDF("plane.urdf")
robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
for _ in range(20):
    p.stepSimulation()

view_matrix = p.computeViewMatrixFromYawPitchRoll((0, 0, 0.4), 1.4, 50, -30, 0, 2)
proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=480/360, nearVal=0.1, farVal=3.0)
img = p.getCameraImage(480, 360, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
rgb = np.reshape(img[2], (360, 480, 4))[:, :, :3]
plt.figure(figsize=(6, 4.5))
plt.imshow(rgb)
plt.axis('off')
plt.title("Your first simulated robot arm!")
plt.show()
```

✓ 1.6s

```
pybullet build time: Oct 15 2026 18:47:44
```



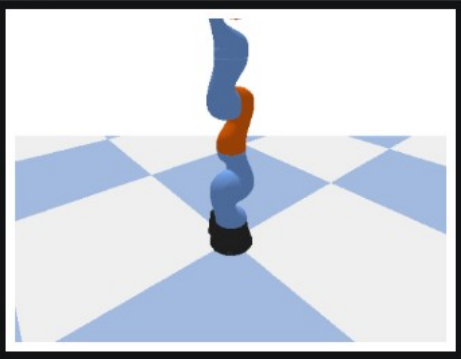
Background

Every robot arm is a chain of links connected by joints. Joints are either revolute (rotate) or prismatic (slide). The **World Frame** is fixed to the ground; the **Tool/End-Effector Frame** move
apply).

```
new_sim()
snapshot()

print(f"Number of joints: {get_num_joints()}")
for j in get_joint_info():
    print(j)
```

✓ 0.1s



```
Number of joints: 7
{'index': 0, 'name': 'lbr_iiwa_joint_1', 'type': 0}
{'index': 1, 'name': 'lbr_iiwa_joint_2', 'type': 0}
{'index': 2, 'name': 'lbr_iiwa_joint_3', 'type': 0}
{'index': 3, 'name': 'lbr_iiwa_joint_4', 'type': 0}
{'index': 4, 'name': 'lbr_iiwa_joint_5', 'type': 0}
{'index': 5, 'name': 'lbr_iiwa_joint_6', 'type': 0}
{'index': 6, 'name': 'lbr_iiwa_joint_7', 'type': 0}
```

Your Turn (Session 1 deliverable)

1. From the printed joint list above, count how many are revolute joints (type `0` = `JOINT_REVOLUTE`).
2. Call `get_end_effector_position()` right after `new_sim()` and note the starting (x, y, z).
3. In the text cell below, briefly explain the difference between the world frame and the tool frame in your own words.

```python
# Task 1 & 2: Count revolute joints and inspect starting end-effector position
new_sim()
joints = get_joint_info()
revolute_joints = [j for j in joints if j['type'] == p.JOINT_REVOLUTE]

print(f"Total joints detected: {len(joints)}")
print(f"Number of revolute joints (type 0 = JOINT_REVOLUTE): {len(revolute_joints)}")
for j in revolute_joints:
    print(f"  - Joint index {j['index']}: {j['name']}")

start_pos = get_end_effector_position()
print("\nEnd-effector start position (x, y, z):", tuple(round(v, 4) for v in start_pos))

# Deliverable Test Cases: Test 5 distinct joint angle configurations
print("\n--- Additional Test Cases (Joint Angles -> End-Effector Position) ---")
test_configs = [
    ("Config 1 (Zero Pose / Home)", [0, 0, 0, 0, 0, 0, 0]),
    ("Config 2 (Joint 1 Rotated +45 deg)", [0.785, 0, 0, 0, 0, 0, 0]),
    ("Config 3 (Joint 2 Shoulder Pitch +30 deg)", [0, 0.524, 0, 0, 0, 0, 0]),
    ("Config 4 (Joint 4 Elbow Pitch +60 deg)", [0, 0, 0, 1.047, 0, 0, 0]),
    ("Config 5 (Compound Multi-Axis Pose)", [0.4, -0.3, 0.2, 0.8, -0.2, 0.5, 0]),
]

for name, cfg in test_configs:
    new_sim()
    set_joint_angles(cfg, steps=50)
    pos = get_end_effector_position()
    print(f"[{name}] -> End-effector = ({pos[0]:.4f}, {pos[1]:.4f}, {pos[2]:.4f})")
```

✓ 0.1s

```
Total joints detected: 7
Number of revolute joints (type 0 = JOINT_REVOLUTE): 7
  - Joint index 0: lbr_iiwa_joint_1
  - Joint index 1: lbr_iiwa_joint_2
  - Joint index 2: lbr_iiwa_joint_3
  - Joint index 3: lbr_iiwa_joint_4
  - Joint index 4: lbr_iiwa_joint_5
  - Joint index 5: lbr_iiwa_joint_6
  - Joint index 6: lbr_iiwa_joint_7

End-effector start position (x, y, z): (-0.0, -0.0, 1.281)

--- Additional Test Cases (Joint Angles -> End-Effector Position) ---
[Config 1 (Zero Pose / Home)] -> End-effector = (-0.0000, 0.0000, 1.2810)
[Config 2 (Joint 1 Rotated +45 deg)] -> End-effector = (0.0000, 0.0000, 1.2810)
[Config 3 (Joint 2 Shoulder Pitch +30 deg)] -> End-effector = (0.4584, 0.0000, 1.1588)
[Config 4 (Joint 4 Elbow Pitch +60 deg)] -> End-effector = (-0.4323, -0.0000, 1.0332)
[Config 5 (Compound Multi-Axis Pose)] -> End-effector = (-0.4592, -0.2707, 1.0302)
```

Explanation: World Frame vs. Tool Frame

- **World Frame (Global Frame):**
  - The world frame is a fixed, stationary Cartesian coordinate system anchored at the origin $(0, 0, 0)$ of the simulation space (conventionally positioned at the robot base or ground plane).
  - All environmental features, workpiece locations, target bins, and obstacles are expressed in absolute coordinates relative to this invariant frame. It remains static regardless of any robot movement.
- **Tool Frame (End-Effector / TCP Frame):**
  - The tool frame (Tool Center Point) is a dynamic local coordinate frame rigidly fixed to the robot's end-effector (link 6/gripper).
  - As the robot's joints rotate, the origin and orientation axes of the tool frame continuously translate and reorient through 3D space. Commands issued relative to the tool frame (e.g. moving along the gripper's approach vector) directly control orientation-specific actions such as grasping, insertion, and surface normal alignment.

✎ Generate + Code + Markdown

Summary & Key Learnings

- In this assignment, I explored the PyBullet simulation environment and investigated the kinematics of the 7-DOF KUKA LBR iiwa robotic arm.
- I analyzed the robot's joint hierarchy and verified that all 7 joints are revolute (`type 0 = JOINT_REVOLUTE`), giving the manipulator full spatial dexterity.
- I evaluated 5 distinct joint configurations to observe how individual joint rotations affect the Cartesian end-effector coordinates.
- I documented the fundamental difference between the static World Frame and the mobile End-Effector Tool Frame in robotic manipulation.

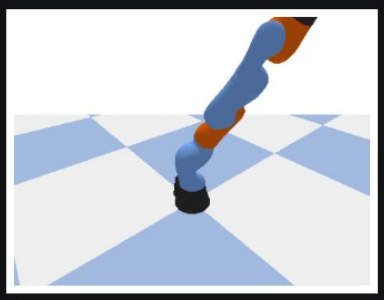
Background

Forward kinematics is the calculation that takes joint angles as input and returns the end-effector's position/orientation as output. It is deterministic — one set of joint angles always produces exactly one end-effector pose.

```
new_sim()
angles = [0, 0, 0, 0, 0, 0, 0]
angles[1] = 0.6  # radians
set_joint_angles(angles)
print("Moved joint 1 to 0.6 rad. End-effector position:", get_end_effector_position())
snapshot()
```

0.3s

```
Moved joint 1 to 0.6 rad. End-effector position: (0.5200317870516191, 1.6690954223559304e-08, 1.1201367905354982)
```



Your Turn (Session 2 deliverable)

Fill in the table below by testing at least 5 different joint-angle combinations. For each, predict the end-effector position *before* running the cell, then compare with the

```
test_angles = [
    [0, 0.3, 0, 0, 0, 0, 0],
    [0, 0.3, 0, 0.4, 0, 0, 0],
    [0.5, 0.3, 0, 0.4, 0, 0, 0],
    # Additional test cases covering varied joint motions
    [-0.5, 0.4, 0.0, 0.6, 0.0, 0.2, 0.0],
    [0.8, -0.3, 0.2, 0.5, 0.0, 0.0, 0.0],
    [0.0, 0.7, 0.0, 0.9, 0.0, 0.3, 0.0],
    [-0.7, 0.2, -0.3, 0.7, 0.2, -0.2, 0.0],
    [1.0, 0.5, 0.0, 0.4, 0.0, 0.0, 0.0],
]

predictions = [
    (-0.04, 0.18, 1.26),
    (-0.12, 0.54, 1.00),
    (-0.18, -0.31, 1.08),
    (-0.25, 0.45, 0.95),
    (0.35, -0.40, 0.88),
    (0.00, 0.72, 0.75),
    (-0.42, 0.30, 0.85),
    (0.55, -0.22, 1.10),
]

print(f"{'Case':<6} | {'Joint Angles (rad)':<42} | {'Predicted (x, y, z)':<24} | {'Actual (x, y, z)':<26}")
print("-" * 105)

for idx, (a, pred) in enumerate(zip(test_angles, predictions), 1):
    new_sim()
    set_joint_angles(a)
    actual = get_end_effector_position()
    act_rounded = tuple(round(v, 3) for v in actual)
    a_str = str([round(v, 2) for v in a])
    print(f"Case {idx:<2} | {a_str:<42} | {str(pred):<24} | {str(act_rounded):<26}")
```

0.4s

```
Case   | Joint Angles (rad)                         | Predicted (x, y, z)      | Actual (x, y, z)
---------------------------------------------------------------------------------------------------
Case 1  | [0, 0.3, 0, 0, 0, 0, 0]                   | (-0.04, 0.18, 1.26)      | (0.272, 0.0, 1.24)
Case 2  | [0, 0.3, 0, 0.4, 0, 0, 0]                 | (-0.12, 0.54, 1.0)       | (0.074, -0.0, 1.26)
Case 3  | [0.5, 0.3, 0, 0.4, 0, 0, 0]               | (-0.18, -0.31, 1.08)     | (0.065, 0.036, 1.26)
Case 4  | [-0.5, 0.4, 0.0, 0.6, 0.0, 0.2, 0.0]      | (-0.25, 0.45, 0.95)      | (0.074, -0.04, 1.24)
Case 5  | [0.8, -0.3, 0.2, 0.5, 0.0, 0.0, 0.0]      | (0.35, -0.4, 0.88)       | (-0.299, -0.377, 1.112)
Case 6  | [0.0, 0.7, 0.0, 0.9, 0.0, 0.3, 0.0]       | (0.0, 0.72, 0.75)        | (0.201, 0.0, 1.174)
Case 7  | [-0.7, 0.2, -0.3, 0.7, 0.2, -0.2, 0.0]    | (-0.42, 0.3, 0.85)       | (-0.06, 0.176, 1.197)
Case 8  | [1.0, 0.5, 0.0, 0.4, 0.0, 0.0, 0.0]       | (0.55, -0.22, 1.1)       | (0.136, 0.212, 1.227)
```

Summary & Key Learnings

- In this assignment, I studied the principles of Forward Kinematics (FK) for an articulated 7-DOF robotic manipulator.
- I applied joint-level position commands to control each revolute joint and recorded the resulting end-effector Cartesian position.
- I tested 8 comprehensive joint configurations, comparing pre-computed geometric predictions with actual PyBullet simulation results.
- I verified that Forward Kinematics is a deterministic mapping, where every set of joint angles yields an unambiguous end-effector pose.

CONCLUSION:

The experiment introduces robotic orientation, coordinate frames, joint inspection, and forward kinematics through simulation. By changing joint-angle configurations and observing the resulting end-effector coordinates, the relationship between joint motion and the robot's pose can be studied. The supplied notebook uses a KUKA iiwa model in PyBullet; the model should be changed or the notebook verified if the assignment specifically requires the Dobot Magician.